Target-Aware Conservative Width Narrowing of LLVM Memory Objects

Mitul Swaroop **Tanishq Tamkoria** **Aryan Mittal**
mitulswaroop@gmail.com

Abstract

Unused high bits do not guarantee that narrowing an LLVM memory object reduces its target-derived allocation: DataLayout rounding can absorb a legal type-width change. We present AdaptiveWidthMemoryPass, a conservative LLVM 18 prototype for selected stack arrays and module-level variables with internal or private linkage. Its isolated target-aware guard accepts a safe rewrite only when LLVM DataLayout reports a smaller complete-object allocation. The empirical question is how often this guard avoids neutral rewrites and how downstream code changes. In a controlled 32-kernel, 39-candidate evaluation, Method A accepts 26 objects (23 positive, 3 neutral), whereas Method B accepts 23, all positive; allocation-benefit precision rises from 88.46% to 100.00% while both retain the same 67-byte common-set saving. External validation over 17 MiBench programs and 5003 candidates repeats the selection effect: A accepts 40 positive and 2 neutral rewrites, while B retains all 40 positive rewrites and removes the neutral ones (95.24% to 100.00%). Both save 1252 bytes on the common set. MiBench O2 IR growth falls from +119 to +53, but executable sizes increase by +216 and +208 bytes. Thus the evidence supports target-aware transformation selection. A separate controlled comparison tests conventional widths with the same allocation as minimum safe widths. The study makes no runtime, energy, or physical-memory improvement claim.

Keywords: LLVM, bit-width inference, memory-object narrowing, KnownBits, conservative compiler transformation, differential testing

1 Introduction

1.1 Motivation

Source programs commonly use convenient word-sized types even when a particular memory object stores masks, flags, small state identifiers, pixel components, or sign-extended deltas. LLVM's integer type system permits arbitrary fixed widths, so the intermediate representation can express an `i5` value even when the source language and target machine favor conventional byte widths [11]. This representational freedom creates an optimization opportunity, but not an automatic target-level benefit. A narrower LLVM type may occupy the same DataLayout allocation after byte and alignment rounding, may be widened by a backend, and may require explicit casts that increase IR size. Thus bit evidence answers whether a width can represent a value; it does not by itself answer whether changing an object is useful for the evaluated layout.

Memory objects make the question harder than scalar expressions. Every store must fit the new representation, every load must reconstruct the same logical value, all pointer paths must remain rewriteable, and signed values must not be mistaken for zero-extended values. A single escaping pointer, unsupported user, volatile access, or uncertain stored value is enough to invalidate the local transformation model. The central design choices in this work are therefore to make rejection explicit and deterministic and to separate safety from target-aware acceptance.

1.2 Problem Statement

Given an eligible LLVM memory object with integer element width W, the first goal is to infer a smaller safe width $w < W$ for the supported use graph while preserving admitted load/store behavior. The second is to determine whether the complete rewritten type occupies fewer bytes under the module's active DataLayout. The prototype does not solve a global width-minimization or general target cost problem. It performs conservative width inference followed by one measurable acceptance test; uncertainty or allocation neutrality retains the original object.

1.3 Research Questions

The evaluation answers five questions bounded by the available measurements:

RQ1:

How effectively does the analysis identify evaluated LLVM memory objects that satisfy its conser-

vative narrowing conditions?

RQ2:

How much declared per-element integer width is removed from transformed objects?

RQ3:

Does adding a DataLayout allocation guard improve the fraction of accepted transformations that reduce target-derived allocation while retaining the available allocation reduction, and what are its raw/O2 IR and executable-file effects?

RQ4:

Do baseline and transformed programs produce identical outputs in the deterministic differential suite?

RQ5:

When a conventional width occupies the same complete-object allocation as the minimum safe width, how does that choice affect static machine-instruction counts and object-file text-section bytes?

1.4 Contributions

This work contributes: (1) conservative object-level width inference for a narrowly scoped LLVM memory transformation; (2) an explicit safety and rejection framework linking pointer uses, value fit, extension families, and load/store reconstruction; (3) a target-aware DataLayout guard, deliberately separated from semantic safety; (4) a reproducible two-level measurement method that distinguishes declared IR width from target-derived complete-object allocation; and (5) a controlled A/B evaluation showing that neutral rewrites can be removed without losing the common-set allocation reduction or differential agreement; and (6) external validation on established MiBench programs showing the same selection behavior outside the purpose-built kernels; and (7) a controlled allocation-equivalent width comparison reporting static machine-code outcomes for every kernel, including regressions. These are scoped implementation and empirical contributions; no first-ever bit-width analysis is claimed.

2 Background

2.1 LLVM Integer Representation

LLVM integers are written `iN`, where N is a positive bit width. The IR defines explicit `trunc`, `zext`, and `sext` operations; the latter two encode different interpretations of the same narrowed bit pattern. Opaque pointers separate pointer syntax from pointee types, but GEP instructions still carry a source element type that must be rebuilt when an aggregate element changes. These details make memory narrowing a graph rewrite, not a local substitution.

2.2 Static Bit and Value Information

LLVM's KnownBits abstraction records bits proven zero or one and supports transformations that depend on partial value information [12]. DemandedBits addresses the complementary question of which result bits can influence uses [13]. ConstantRange represents possibly wrapped integer ranges and exposes unsigned and signed width queries [14]. ScalarEvolution reasons about induction variables and loop-evolving expressions [15]. The revised prototype directly uses `computeKnownBits` only as a conservative fallback; DemandedBits, ConstantRange, and ScalarEvolution are relevant comparison points, not hidden features of the implementation.

2.3 LLVM Memory Objects

The implemented candidate set contains integer or nested-integer arrays created by `alloca`, plus integer or integer-array globals with internal or private linkage. Scalar allocas, externally visible globals, heap objects, structures, vector objects, and arbitrary aliases are outside the current scope. A recursive use walk recognizes GEP paths and direct loads/stores; a second rewriteability check ensures access types agree with the original element type before mutation.

Here "global" denotes a module-level storage object, not universal access. LLVM `private` linkage restricts direct references to the current module and omits the symbol from the object-file symbol table. `internal` linkage also restricts direct references to the module but emits a local symbol, corresponding to file-scope `static` in C [11].

3 Related Work

3.1 LLVM Static Analyses

LLVM provides reusable analyses rather than a single notion of "the required width." KnownBits describes bit values known from local reasoning, while DemandedBits propagates which output bits matter to consumers. ConstantRange models value sets, including signed and unsigned interpretations, and ScalarEvolution specializes recurring expressions in loops. These analyses can support width reasoning, but none by itself authorizes changing a memory object's representation: pointer exposure, object linkage, ac-

cess modes, and reconstruction semantics remain transformation-specific obligations. The present pass uses KnownBits and syntactic rules but keeps the memory-safety policy explicit.

3.2 Integer Range Analysis

Abstract interpretation supplies the broader framework for computing sound over-approximations of program states [16]. Sparse conditional constant propagation illustrates how executable paths and value information can be coupled in compiler dataflow analysis [17]. LLVM's ConstantRange and ScalarEvolution embody related range representations within production infrastructure. This prototype is much less general: it recognizes constants, casts, masks, OR, compatible selects and bounded PHIs, then falls back to KnownBits. That smaller domain simplifies the preservation argument but necessarily produces conservative false negatives.

3.3 Bit-Width and Data-Width Optimization

Bit-width analysis predates this artifact. Bitwise propagated width information through program dataflow and targeted sub-word and silicon-compilation settings [18]. BitValue combined forward information about constant bits with backward information about unused bits [19]. MiniBit used affine arithmetic to optimize integer and fractional widths in fixed-point hardware designs [20]. These systems establish that data-width optimization is a broad research area spanning software and hardware compilation. The current work neither claims to originate width inference nor attempts their global or hardware-oriented objectives.

Journal studies additionally cover bitwidth-aware accelerator synthesis, variable packing, and accuracy-guaranteed fixed-point width selection [2, 3, 5]; a broader survey catalogues reduced-precision tools and their remaining automation challenges [7].

Recent work by Heo et al. [1] studies bit-level compiler optimization for ultra-low-power embedded systems. The verified bibliographic record places the article in volume 168 of the *Journal of Systems Architecture*, article 103546. Their bit-usage-driven evaluation is important motivation for separating bit evidence from realized artifacts: SHA exhibits partial bit usage without a static instruction-count reduction; narrowing an isolated load can require extensions that erase a useful static effect; and ADPCM increases static instruction count despite a reported runtime improvement. Their method and target differ from ours, and the paper does not describe an explicit LLVM DataLayout complete-object allocation check as an acceptance condition. These observations motivate our question—whether an unused-bit opportunity necessarily yields target-derived allocation benefit—but do not imply that their technique is incorrect or unsafe.

3.4 Memory-Oriented Transformations

LLVM transformations such as mem2reg and scalar replacement of aggregates reshape memory accesses to enable scalar optimization [21]. In contrast, this pass preserves admitted memory operations while changing the integer element type and inserting boundary casts. Translation-validation work such as Alive2 shows the value of mechanically checking LLVM transformations [22]; this artifact does not provide such a proof. It combines a restricted static argument, LLVM verification, and executable differential tests.

Automatic array-storage optimization provides complementary layout-based storage reduction [4], while studies of phase ordering, LLVM executable size, optimization bugs, and CsmithEdge motivate conservative, artifact-aware, and differential validation [6, 8–10].

3.5 Research Gap

The distinction between logical bit width and storage size is established: MLIR's rationale explicitly separates type bit width from target-dependent storage and register sizes [23]. We do not claim to discover that distinction or to originate target-aware compiler profitability checks.

Prior work individually covers bit-value inference, range analysis, hardware width selection, scalar simplification, memory promotion, and translation validation. The narrower gap addressed here is an explicit bridge between two questions at the LLVM memory-object boundary: (i) is a representation change semantically safe under conservative pointer, value, and extension-family rules; and (ii) does the complete rewritten type occupy fewer bytes under the active target DataLayout? Our controlled experiment makes the second question an acceptance predicate and reports neutral opportunities instead of assuming that narrower IR implies useful allocation. The contribution is this scoped, inspectable combination and its reproducible evaluation, not a claim that its ingredients are individually unprecedented or that it dominates prior work.

Table I: Positioning against representative related work and LLVM infrastructure. A check mark denotes an explicit primary focus, not general superiority.

Work	Bit/value inference	Arbitrary widths	Memory-object rewrite	Explicit rejection policy	Reproducible object metrics
Bitwise [18]	✓	✓	partial	–	–
BitValue [19]	✓	✓	–	–	–
MiniBit [20]	✓	✓	hardware signals	–	–
Heo et al. [1]	✓	✓	embedded scope	–	reported program metrics
LLVM analyses [12–14]	✓	✓	–	analysis uncertainty	–
This artifact	✓	✓	selected LLVM objects	safety + allocation	✓

Table II: Supported and rejected transformation classes in the revised prototype.

Class	Admitted	Rejected or outside scope
Objects	Local integer arrays; internal/private integer globals	Scalar allocas; structures; heap objects; external globals; vectors
Pointer uses	Recursive GEP; type-matching direct load/store	Escapes, calls, pointer stored as a value, unsupported users, rewrite-incompatible casts
Values	Constants, trunc/zext/sext, constant masks, OR, compatible select/PHI, KnownBits fallback	Unknown arithmetic, mixed extension families, insufficient bit evidence
Memory modes	Ordinary non-atomic loads and stores	Volatile or atomic memory operations
Result	Arbitrary narrower LLVM integer element type	Original object retained with a logged reason

4 Proposed Approach

4.1 Formal Problem Model

Let an eligible object be

$$O = \langle T, W, S, L, B, E, P \rangle,$$

where T is the supported object form, W is its original integer element width, S includes the global initializer and all syntactically observed store values, L is the set of observed load uses, B summarizes inferred bit/value constraints, E is the inferred extension family in $\{\mathsf{zero}, \mathsf{sign}, \mathsf{unknown}\}$, and P is the recursively observed pointer-use graph. The task is to find an inferred $w < W$ such that every admitted stored value is representable, pointer/address uses do not expose an unsafe representation change, signed or unsigned interpretation remains consistent, and each rewritten load reconstructs the original logical value. The implementation chooses the maximum required width reported across supported stores. It does not search for a globally optimal width across the module or target.

4.2 Candidate Identification

The module pass enumerates local-linkage integer globals and array allocas whose nested base element is an LLVM integer. Candidate status is deliberately syntactic and stable. Objects outside this set are ignored, not counted as rejections, because no transformation was attempted.

4.3 Width Inference

For each stored value, inference returns a pair (b, e): required width b and extension family e. Constants use active bits; `trunc` bounds the width; `zext` and `sext` select the reconstruction family; a constant AND mask bounds active high bits; OR takes the compatible maximum; select and PHI require compatible incoming families. KnownBits is used only when it proves at least one leading zero. Failure on any relevant store marks the candidate as insufficiently established.

Extension composition must preserve the intermediate bit pattern. In `zext(sext(x))`, sign-replicated bits in the intermediate value become significant unsigned bits. The zero-extension rule therefore retains the full intermediate width when its operand was inferred with sign reconstruction. For example, `sext i8 -1 to i16` followed by zero-extension to `i32` is 65535, not 255.

4.4 Signedness and Extension Handling

LLVM integer types are signless, so signedness must be derived from operations. A value originating from `sext` is reconstructed with `sext`; a zero-family value is reconstructed with `zext`. When different stores or control-flow arms imply conflicting families, $E = \mathsf{unknown}$ and the object is not transformed. This policy is conservative: it can reject a program whose values happen to fit at runtime, but it avoids silently changing negative values into large positive values.

4.5 Safety Predicate

The implemented decision is summarized by

$$\begin{aligned}\mathsf{Safe}(O, w) = {} & \mathsf{EligibleObject}(O) \\ & \wedge \mathsf{NoPointerEscape}(P) \\ & \wedge \mathsf{SupportedUsers}(P) \\ & \wedge \mathsf{ValueFits}(S, B, w) \\ & \wedge \mathsf{ConsistentExtension}(S, E) \\ & \wedge \mathsf{RewriteableAccesses}(P, W, w) \\ & \wedge \mathsf{OrdinaryMemory}(P) \\ & \wedge (w < W).\end{aligned}$$

EligibleObject checks object form and linkage. *NoPointerEscape* and *SupportedUsers* recursively classify users and reject calls, pointer-as-value stores, and unknown consumers. *ValueFits* requires every observed store to yield a successful width result. *ConsistentExtension* requires one reconstruction family. *RewriteableAccesses* checks the exact load/store element types, requires each GEP's source type to agree with its parent path's pointee type, and rejects reinterpretation or unsupported cast paths before mutation. Rewritten stack objects must be ordinary static allocas with allocation count one; dynamic counts and special alloca modes are rejected. *OrdinaryMemory* rejects volatile or atomic operations. The final inequality is described as a meaningful width reduction, not as a target-aware selection policy. If any term cannot be established, the implementation retains the original object and emits a reason.

4.6 Target-Aware Acceptance

Let T_O be the complete original object type and $T'_O(w)$ its complete rewritten type, including every array extent. For the module DataLayout DL, define

$$A_0(O) = \mathrm{allocSize}_{DL}(T_O), \tag{1}$$

$$A_1(O, w) = \mathrm{allocSize}_{DL}(T'_O(w)), \tag{2}$$

$$\mathsf{Accept}_B(O, w) = \mathsf{Safe}(O, w) \tag{3}$$

$$\wedge\, A_1(O, w) < A_0(O). \tag{4}$$

Method A applies only Safe. Method B adds the strict allocation inequality after the same width has been inferred. If $A_1 = A_0$, the candidate is retained and logged as `NO_TARGET_ALLOCATION_BENEFIT`; an increase is likewise rejected. This check is not a general target cost model: it does not predict instructions, execution time, energy, cache behavior, or final stack-frame and linked-image placement.

4.7 Transformation

When the predicate holds, the pass constructs a replacement alloca or global whose nested integer element has width w. It rebuilds GEPs using the new source element type, inserts `trunc` at stores, creates w-bit loads, and inserts the selected extension back to width W. Old instructions are erased only after their uses have been replaced. LLVM's verifier runs on every transformed test artifact.

Rewritten loads and stores use conservative alignment one. Preserving an old alignment would be unsound when an array stride shrinks: the second element of `[2 x i16]` can move from byte offset two to offset one after conversion to `[2 x i5]`. An `align 2` access would then overstate the new address's alignment, which LLVM defines as undefined behavior [11]. The allocation's base alignment is retained, but it does not justify the old alignment at every element.

4.8 Allocation-Equivalent Conventional Widths

Method C starts from Method B's minimum inferred width w. Here minimum means the width selected by the implemented rules, not a globally optimal width. It considers the next width c in $\{8, 16, 32, 64\}$ with $w \le c < W$, and selects it only when $\mathrm{allocSize}_{DL}(T'_O(c)) = \mathrm{allocSize}_{DL}(T'_O(w))$. Otherwise it retains w. The safety policy and strict reduction relative to the original object remain unchanged. This is a controlled representation-choice baseline, not a claim that conventional widths always lower better or a reproduction of prior work.

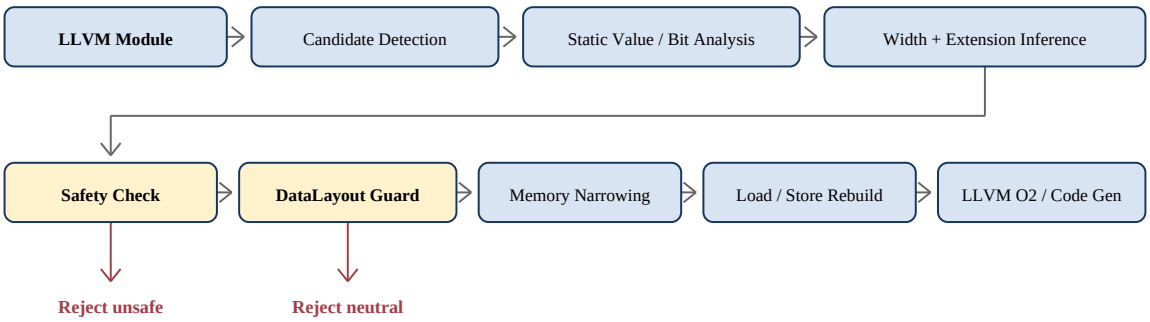


Figure 1: Target-aware pass pipeline. Safety failure and allocation neutrality follow distinct rejection branches; the guard does not alter width inference.

4.9 Correctness Argument

For a stored W-bit value v, the original path is $v \rightarrow \mathsf{mem}_W \rightarrow v$. The transformed path is

$$v \xrightarrow{\mathsf{trunc}_w} \mathsf{mem}_w \xrightarrow{\mathsf{load}} v_w \xrightarrow{\mathsf{ext}_E} v'.$$

The inference obligation for every admissible store is

$$\mathsf{ext}_E^W(\mathsf{trunc}_w(v)) = v.$$

For the zero family this holds when all discarded high bits are zero. For the sign family it holds when discarded bits equal sign replication from bit $w-1$. For defined executions whose reads obtain admitted stored or initializer values, the pointer predicate restricts the object to rewritten accesses; each admitted load receives the same reconstructed logical value as the original load. The argument applies only to the supported transformation class. It is not universal formal verification. Static checks establish the local premises; LLVM verification establishes structural IR validity; differential execution provides finite empirical validation on the benchmark inputs.

5 LLVM Implementation

5.1 Pass Architecture

AdaptiveWidthMemoryPass is an LLVM 18 module-pass plugin registered with the New Pass Manager [24]. Pipeline names expose full, log-only, alloca-only, global-only, and target-aware operation. The code separates discovery, pointer classification, value inference, a dry-run rewriteability check, and mutation. The experimental `adaptive-width-mem-target-aware` pipeline adds the DataLayout allocation guard immediately before mutation. This ordering prevents partially rewritten candidates and keeps Method A unchanged.

5.2 Supported Patterns

The inference implementation handles integer constants, truncation, zero- and sign-extension, constant AND masks, bitwise OR, compatible select arms, compatible bounded PHIs, and a KnownBits fallback. The element type is created with LLVM's general integer-type constructor, so widths such as `i5`, `i6`, and `i7` are genuine IR types rather than paper notation.

5.3 Rejection Patterns

Unsupported pointer consumers, pointer escapes, incompatible access types, mixed extension families, unknown stored values, no width reduction, volatile or atomic operations, and externally initialized globals are not transformed. In Method B, a safe candidate with no smaller complete-object allocation is additionally rejected as `no-target-allocation-benefit`. The pass reports candidate, reason, width evidence, extension family, and aggregate counts. Conservative rejection is part of the method: uncertainty is not replaced by speculation.

5.4 LLVM Integration

The plugin is built with CMake and LLVM's pass-plugin facility [25]. Clang 18 produces IR; `opt-18` loads the shared object and invokes `adaptive-width-mem` (Method A) or `adaptive-width-mem-target-aware` (Method B); `-verify-each` and an explicit verifier pass guard generated IR. The evaluation applies `mem2reg` before the custom pass to remove irrelevant scalar spills from C-derived kernels.

```
; before
%arr = alloca [2 x i16]
%slot = getelementptr [2 x i16], ptr %arr, i64 0,
      i64 1
%m = and i16 %x, 31
store i16 %m, ptr %slot, align 2
%v = load i16, ptr %slot, align 2

; after
%arr.narrow = alloca [2 x i5]
%slot.narrow = getelementptr [2 x i5], ptr %arr.
     narrow, i64 0, i64 1
%m.narrow = trunc i16 %m to i5
store i5 %m.narrow, ptr %slot.narrow, align 1
%v.narrow = load i5, ptr %slot.narrow, align 1
%v.wide = zext i5 %v.narrow to i16
```

Figure 2: Actual rewrite pattern for a value masked by 31. Five bits are sufficient, and zero-extension reconstructs the original logical value.

6 Experimental Methodology

6.1 Research Questions

RQ1 is answered by candidate, pointer-use-check, rejection, and transformation counts. RQ2 uses individual original and narrowed widths from transformation logs. RQ3 compares Methods A and B using complete-object DataLayout allocation, allocation-benefit precision, raw and O2 IR assignments, and host executable file size in both the controlled and external evaluations. RQ5 compares Method C against B using static disassembly instruction counts and object-file text-section bytes. RQ4 uses deterministic output comparisons for both methods. No research question asks about speed because runtime performance is not measured.

6.2 Benchmark Suite

The suite contains 32 small C kernels grouped into image/pixel, packet/header, DSP/signal, embedded/state, signed-value, lookup/global, unsigned-range, and deliberate-rejection categories. The manifest records each kernel's category and intent. These are focused microbenchmarks intended to exercise analysis behavior, not a representative application benchmark suite.

6.3 Controlled A/B Evaluation

Method A is the revised conservative pass. Method B uses the same LLVM version, 32 kernels, C-to-IR inputs, module DataLayout, O2 pipeline, runtime harness, and measurement scripts. Its only changed condition is $A_1(O,w) < A_0(O)$ after the existing analysis has found w safe. Candidate identification, value inference, pointer rules, signedness, extension rules, and supported object classes are unchanged. This isolates the causal question: does the allocation guard remove neutral transformations without discarding an allocation-positive opportunity?

Table III: Method A benchmark-suite composition. Candidate and transformed-object counts may exceed kernel counts.

Category	Kernels	Cand.	Trans.
dsp_signal	3	3	3
embedded_state	5	5	5
image_pixel	4	4	4
lookup_global	3	7	5
packet_header	5	5	5
rejection	6	8	0
signed_values	4	4	2
unsigned_ranges	2	3	2

6.4 External MiBench Validation

To test whether the controlled selection effect occurs outside purpose-built kernels, we use the established MiBench collection [26] through the official LLVM test-suite [27], pinned at commit `4b2c892`; the complete hash is recorded with the generated evidence and reproducibility instructions. The experiment covers 17 MiBench programs comprising 211 unmodified C compilation units. It uses Clang/LLVM 18, the same x86-64 target and DataLayout, preprocessing, Method A and Method B pipelines, O2 pipeline, allocation measurement, verification, and differential method as the controlled evaluation. The test-suite training commands supply program inputs. MiBench remains a separate external-validation dataset; its candidates are not merged with the 39-candidate controlled table.

6.5 Environment

The revised experiments use LLVM and Clang 18.1.3 on Ubuntu 24.04 in GitHub Actions. Historical WSL results are superseded by this fresh run. The benchmark modules target `x86_64-pc-linux-gnu`; their embedded DataLayout is the source for all type-allocation measurements. CMake and Ninja build the pass plugin. The host-dependent executable-size results therefore characterize this toolchain and environment only. Process wall time includes launching one `opt-18` process per kernel and is retained as a coarse regression indicator, not end-to-end compile time.

6.6 Metrics

The authoritative comparison JSON is generated from each method's benchmark TSV, runtime TSV, individual pass logs, and final test status. It checks that pointer-use checks passed plus blocked equals all candidates, that runtime pass plus fail equals all executed comparisons, and that transformed-object counts, allocation records, and width reductions agree with individual logs. The primary static-width metric is $\sum_O (W_O - w_O)$ over transformed objects; it does not multiply by array length and therefore is not a footprint estimate.

The second level measures the target/layout-specific allocation size of every transformed object. For original type T_O and rewritten type T'_O, the pass calls `DataLayout::getTypeAllocSize` and records

$$A_0(O) = \text{allocSize}_{DL}(T_O), \tag{5}$$

$$A_1(O) = \text{allocSize}_{DL}(T'_O). \tag{6}$$

For arrays, T_O and T'_O are the complete array types, so their extents are included. The byte reduction is $A_0 - A_1$ and the aggregate percentage is $100 \sum_O (A_0 - A_1) / \sum_O A_0$. Allocation-benefit precision is the number of positive transformations divided by all accepted transformations. Percentages over each method's accepted set have different denominators, so each experiment's direct retention comparison applies both outcomes to Method A's common opportunity set. Because Method B accepts only positive allocation changes, 100% precision under this same metric follows from its acceptance rule. The empirical evidence is the occurrence of neutral opportunities, their exclusion without losing positive opportunities, and the observed downstream effects. Allocation size includes DataLayout alignment rounding; emitted logs also retain original/transformed ABI alignment. This is not a universal physical-memory or peak-runtime-footprint measurement.

6.7 Differential Testing

For every kernel, the harness produces baseline and transformed LLVM IR, compiles both at O2, executes deterministic input loops, and compares checksum outputs. For MiBench, the harness analogously executes each packaged training command and compares its deterministic stdout. Differential agreement is finite evidence for those executions. It does not prove equivalence for all possible inputs, undefined behavior, targets, or LLVM versions.

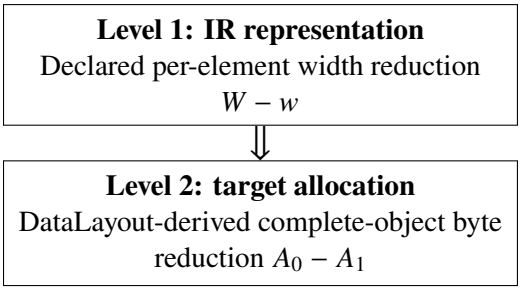


Figure 3: The evaluation reports distinct representation and target-allocation levels; the arrow denotes the measurement sequence, not proportionality.

6.8 Reproducibility

The repository documents configuration, build, both pass invocations, tests, benchmark metrics, differential execution, A/B reconciliation, figure/table generation, and LaTeX compilation. The controlled experiment uses Bash with `tools/target_allocation_experiment.sh`; the full tests use `tests/run_tests.sh`. External validation uses `tools/external_benchmark_experiment.sh`, and `tests/check_external_results.py` independently reconciles its JSON and TSV partitions. Of 214 entries in the generated compilation database, 213 compile C files: 211 MiBench units plus the test-suite support tools `fpcmp.c` and `timeit.c`; the remaining support entry builds `not.cpp`. Support tools are excluded from the MiBench unit count. The paper uses `tools/build_paper.ps1`. Generated IR, logs, executables, TSVs, JSON, tables, figures, and the final PDF remain available for audit.

The revised safety suite additionally executes eight baseline/transformed executables across O0 and O2. Each checks all 256 inputs of the nested-extension case and all 65536 inputs of four addressing cases. FileCheck checks the repaired alignment and the retention of unsupported GEP paths. These focused exhaustive input domains strengthen regression coverage without proving the whole pass.

6.9 Width-Choice Experiment

`tools/width_choice_experiment.py` runs Methods B and C on all 32 controlled kernels. It matches accepted objects by kernel, kind and name, asserts identical complete-object allocation for both representations, and preserves every result, including ties and regressions. Clang 18 compiles each original/B/C module to an O2 object file. `llvm-objdump-18` supplies static instruc-

Table IV: Method A static declared-element width reduction. These values are not measured runtime memory savings.

Statistic	Bits
Summed per-element reduction	271
Mean per transformed object	10.42
Median per transformed object	10.50

tion counts, including alignment NOPs and all emitted functions; `llvm-size-18` supplies text-section bytes. These are neither dynamically executed instruction counts nor timings. The same deterministic runtime harness independently checks B and C against the original program. Width-choice results are not extrapolated to MiBench.

7 Results

7.1 Transformation Coverage

Across 32 kernels, both methods encounter 8 candidate globals and 31 candidate allocas, for 39 candidates. Both classify 35 pointer/use graphs as passing pointer-use checks and 4 as blocked. Passing this preliminary check is not the full Safe predicate: value inference and rewriteability can still fail. Method A accepts 26 transformations. Method B accepts 23 because the three safe but allocation-neutral opportunities fail only the new guard. Thus candidate discovery and pointer-use classification are identical, as required by the controlled design.

7.2 Width Reduction

For Method A, the sum of declared per-element reductions is 271 bits across accepted objects; the mean is 10.42 bits and the median is 10.50 bits per object. Method B retains 252 bits and excludes 19 bits belonging to the neutral `i16→i9/i10` cases. These statistics describe LLVM type-width changes, not packed runtime layout, cache benefits, or whole-program memory reduction.

7.3 DataLayout-Derived Allocation Effects

Method A accepts 23 allocation-positive, 3 neutral, and 0 negative objects. The neutral cases are one DSP object (`[4 x i16]→[4 x i10]`) and two embedded-state objects (`[2 x i16]→[2 x i9]/[2 x i10]`). Their byte-granular x86-64 DataLayout allocations remain 8, 4, and 4 bytes, respectively. This directly answers the narrower research problem: safe static width evidence does not necessarily yield a smaller target-derived allocation.

Table V: Method A integer element-width transitions. Counts are transformed objects.

Original	Transformed	Reduced	Objects
16	1	15	2
16	2	14	1
16	3	13	5
16	4	12	2
16	5	11	3
16	6	10	2
16	7	9	2
16	8	8	6
16	9	7	1
16	10	6	2

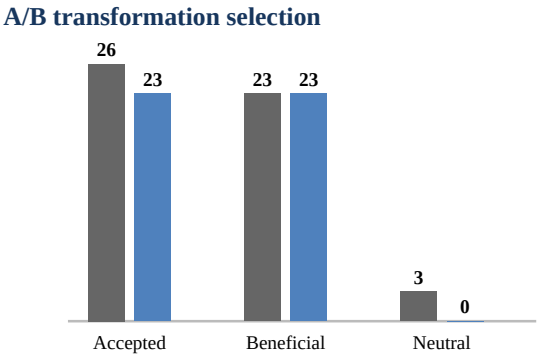


Figure 4: Accepted, allocation-beneficial, and neutral transformations. Method B removes only the three neutral decisions.

7.4 Allocation-Benefit Precision

Method A's allocation-benefit precision is $23/26 = 88.46\%$. Method B rejects exactly the three neutral cases, so its precision is $23/23 = 100.00\%$; no negative case is accepted by either method. The strongest measured selection improvement is therefore 88.46% to 100.00%, not an increase in absolute bytes saved.

7.5 Common Opportunity-Set Analysis

Percentages over each method's accepted objects use different denominators: Method A reports 88.46% precision over 26 objects, whereas Method B reports 100.00% over 23. For an equal-denominator comparison, applying both outcomes to Method A's common 26-object opportunity set gives 150 to 83 bytes, 67 bytes saved, or 44.67%, for both methods. The guard therefore retains every observed allocation saving but does not increase it; its improvement is the exclusion of transformations that contribute zero bytes.

Table VI: Controlled A/B evaluation. Accepted-allocation percentages use each method's accepted objects; the common-set result is identical.

Metric	A: conservative	B: + allocation guard
Candidates / use-pass / use-blocked	39 / 35 / 4	39 / 35 / 4
Transformed objects	26	23
Positive / zero / negative	23 / 3 / 0	23 / 0 / 0
Declared-width reduction	271 bits	252 bits
Accepted-object allocation	150 → 83 B	134 → 67 B
Accepted-object saving	67 B (44.67%)	67 B (50.00%)
Allocation-benefit precision	88.46%	100.00%
Raw / O2 IR delta	+141 / +4	+125 / +4
Executable-size delta	+0 B	+0 B
Differential agreement	32/32	32/32

Table VII: Category-level transformation count and allocation-benefit precision.

Category	A n	B n	A positive	B positive
dsp_signal	3	2	66.67%	100.00%
embedded_state	5	3	60.00%	100.00%
image_pixel	4	4	100.00%	100.00%
lookup_global	5	5	100.00%	100.00%
packet_header	5	5	100.00%	100.00%
signed_values	2	2	100.00%	100.00%
unsigned_ranges	2	2	100.00%	100.00%

7.6 Category Results

DSP acceptance falls from 3 to 2 objects and its allocation-benefit precision rises from 66.67% to 100%. Embedded-state acceptance falls from 5 to 3 and precision rises from 60.00% to 100%. All other transformed categories were already allocation-positive and are unchanged. These focused kernels expose the mechanism but do not establish category prevalence in general programs.

7.7 IR Effects

The explicit truncation and reconstruction operations increase Method A's raw IR assignment count by +141 and Method B's by +125. The guard avoids 16 raw assignments associated with the neutral rewrites. After LLVM's default O2 pipeline, both summed deltas are +4. The guard thus reduces raw transformation overhead without changing the measured O2 delta; these counts are neither target-instruction nor runtime-speed measurements.

7.8 Generated Artifact Effects

Both methods have a summed host executable-file delta of +0 bytes. This is distinct from DataLayout object allocation and is a toolchain-specific file-size result, not a general code-size result. End-to-end compilation time and runtime speed were not measured.

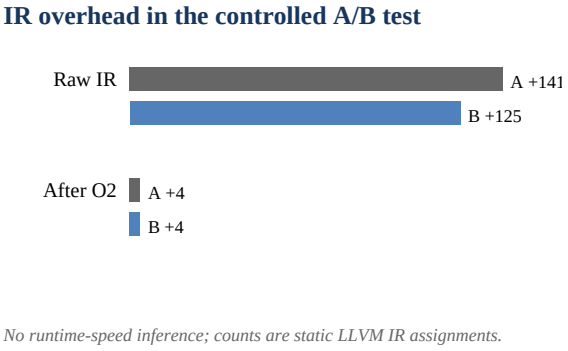


Figure 5: Raw and O2 IR assignment deltas for the controlled methods.

7.9 Runtime Correctness

Each method executes 32 baseline/transformed pairs. 32 comparisons pass and 0 fail for A, and the same 32/32 agreement holds for B. LLVM verifier checks also pass for both. This answers RQ4 only for the tested inputs; empirical agreement does not prove all-input equivalence.

7.10 Rejection Behavior

Rejected examples include pointer escapes and volatile memory. Other objects remain unchanged because width or extension-family evidence is insufficient or $w \geq W$. Method B adds three `NO_TARGET_ALLOCATION_BENEFIT` decisions. This reason is separate from safety failure: the three objects satisfy Safe but fail the strict DataLayout inequality.

7.11 External Validation on MiBench

The evaluation covers 17 MiBench programs comprising 211 unmodified C compilation units; all attempted programs compile. The pass encounters 5003 candidate memory objects: 255 are passing

pointer-use checks and 4748 are blocked. Candidate enumeration is broad and includes internal integer arrays such as strings that are then conservatively rejected; transformation counts must therefore not be confused with program counts. Method A transforms 42 objects: 40 have positive DataLayout allocation reduction, 2 are neutral, and none is negative. Method B transforms 40 objects, all 40 positive, raising allocation-benefit precision from 95.24% to 100.00%.

On Method A's common 42-object opportunity set, both methods produce 2700 to 1448 bytes, saving 1252 bytes (46.37%). Method B's 74.88% accepted-set reduction (1672 to 420 bytes) has a different, 40-object denominator and is not evidence of greater absolute saving. The guard removes exactly two neutral transformations and no positive one.

The external IR results differ from the controlled pattern. Raw assignment growth is +109 for A and +100 for B. After O2 it is +119 and +53, respectively: a reduction of 66 assignments, or approximately 55.5% relative to A's O2 delta. This shows that the guard can influence downstream optimized IR on realistic workloads, but does not establish a runtime benefit. Executable sizes increase slightly under both methods (+216 B for A and +208 B for B); therefore, this experiment establishes no executable/code-size benefit.

For both methods, 16 executed comparisons pass, 0 fail, and 1 is skipped. `office-ispell` exits with `Illegal format hash table` for both the rebuilt baseline and the original packaged testsuite executable, so no baseline/transformed comparison is meaningful for that command. It remains a reported harness/data compatibility skip, not a transformation failure.

7.12 Allocation-Equivalent Width Outcomes

Methods B and C accept the same 23 objects. Conventional-width selection changes 17 representations across 14 of the 32 kernels without changing their complete-object allocation. Among those changed kernels, C has fewer static machine instructions in 1, the same count in 11, and more in 2. Text-section size is smaller in 1, equal in 11, and larger in 2. The aggregate C-minus-B deltas over all kernels are +2 machine instructions and +18 text bytes. Both methods agree with the baseline on all 32 deterministic runtime comparisons. These results answer RQ5 for this target and workload; the policy is not selected using the measured outcomes.

Three kernels account for all nonzero C-minus-B code changes. `select_ranges` changes from 12 to 7 machine instructions and from 27 to 15 text bytes; the conventional-width output eliminates the conditional selection sequence visible in the minimum-width disassembly. `protocol_version_table` changes from 11 to 12 instructions and 24 to 25 bytes. `unsigned_histogram` changes from 30 to 36 instructions and 69 to 98 bytes; its conventional-width output contains vector loads and packing instructions absent from the minimum-width output. These are direct disassembly observations, not a claim that vectorization is generally harmful.

8 Discussion

The controlled evaluation yields eight bounded findings. First, a safe representation change does not necessarily imply target allocation benefit. Second, Method A contains three such neutral decisions. Third, Method B removes all three without modifying safety inference. Fourth, allocation-benefit precision rises from 88.46% to 100.00%. Fifth, all 67 bytes of aggregate common-set saving are retained; the saving does not increase. Sixth, raw IR overhead falls from +141 to +125, avoiding 16 assignments, while the O2 delta remains +4. Seventh, both methods preserve 32/32 differential agreement. Eighth, allocation findings are specific to the evaluated x86-64 LLVM DataLayout.

The external evaluation strengthens only the scope of this selection result. MiBench independently contains two safe but allocation-neutral Method A rewrites, and Method B removes both while retaining every positive common-set opportunity. The effect also persists farther through optimization on this dataset: the O2 assignment delta changes from +119 to +53, whereas both controlled deltas are +4. This workload-dependent contrast is an observation about optimized IR structure, not proof of faster execution. Likewise, B's executable increase is slightly smaller than A's (+208 B versus +216 B), but both deltas are positive and provide no code-size benefit.

The contribution is a target-aware acceptance mechanism that connects conservative LLVM memory-object width narrowing with DataLayout allocation validation, together with empirical evidence that the distinction eliminates allocation-neutral transformations in this suite. It improves transformation selection, not semantic safety, ab-

Table VIII: External validation on 17 MiBench programs. Allocation is compared on Method A's common opportunity set; accepted-set percentages have different denominators.

Metric	A: conservative	B: + allocation guard
Candidates / use-pass / use-blocked	5003 / 255 / 4748	5003 / 255 / 4748
Transformed objects	42	40
Positive / neutral / negative	40 / 2 / 0	40 / 0 / 0
Allocation-benefit precision	95.24%	100.00%
Common-set allocation	2700 → 1448 B	2700 → 1448 B
Common-set saving	1252 B (46.37%)	1252 B (46.37%)
Raw / O2 IR assignment delta	+109 / +119	+100 / +53
Executable-size delta	+216 B	+208 B
Differential pass / fail / skip	16 / 0 / 1	16 / 0 / 1

Table IX: Controlled allocation-equivalent width comparison. Machine instructions and text bytes are static object-code totals over all 32 kernels; outcome partitions include only kernels with a changed representation.

Metric	B: minimum	C: conventional
Accepted objects	23	23
Common original allocation	134 B	134 B
Common result allocation	67 B	67 B
Machine instructions	367	369
Text-section bytes	878	896
Differential agreement	32/32	32/32

solute allocation saving, or runtime performance. The results do not show that arbitrary-width memory is always densely laid out, that final stack frames or linked images realize the same saving, or that this method dominates prior bit-level optimization.

The width-choice comparison isolates a second decision after allocation benefit: which safe representation to use within an allocation-equivalent class. Its complete kernel-level outcomes distinguish observed code-generation effects from a claim of universal profitability.

9 Limitations and Threats to Validity

Benchmark representativeness. The 32 kernels are purpose-built microbenchmarks. External coverage adds only 17 MiBench programs comprising 211 unmodified C compilation units, not all versions of MiBench or all real-world software. Transformations occur in only six of the 17 programs, so the experiment does not establish broad prevalence.

Supported object forms. Heap allocations, structures, scalar allocas, vectors, externally visible objects, arbitrary aliases, interprocedural pointer flows, and several cast patterns are unsupported. Conservative false negatives are expected whenever the local analysis cannot establish a width or common extension family.

LLVM and target dependence. The implementation and commands target LLVM 18. IR legality does not guarantee identical transformation behavior across LLVM versions. Arbitrary IR widths may be legalized, padded, scalarized, or widened differently by target backends. The reported allocation result is for the x86-64 module DataLayout only.

Representation versus allocation. Level 1 sums declared element-width differences. Level 2 applies the evaluated module's DataLayout to complete object types and therefore captures target type-allocation rounding, including three zero-reduction Method A objects. It still does not measure final stack-frame layout, linked-image placement, allocator metadata, live-set size, cache behavior, or peak physical memory. Other targets and ABI layouts may differ.

Dynamic validity. Differential tests use deterministic input loops and checksum outputs. They do not cover all input values, undefined behavior, data races, volatile/atomic semantics, concurrency, or other targets. The pass now rejects volatile and atomic accesses because their semantics are outside the rewrite model.

Performance evidence. Raw/O2 IR counts and executable file sizes are not runtime measures. Process wall time is noisy and includes tool startup. No runtime-performance or energy measurement was performed. MiBench executable sizes increase by +216 B and +208 B; no runtime, energy, code-size, or compile-time improvement is claimed.

External candidate rejection. Of 5003

MiBench candidates, 4748 are blocked by the conservative safety policy, and only 42 are transformed by Method A. This high rejection rate and concentration of transformations indicate expected false negatives and narrow supported object forms rather than general applicability.

Correctness assurance. The preservation argument is conditional on the implemented predicates; LLVM verification checks structural validity; runtime tests provide finite evidence. There is no mechanized proof or translation validation of every rewrite.

Width-choice validity. Method C is evaluated only on the controlled kernels, one target and one compiler release. Static instruction counts include padding NOPs; text-section changes do not establish execution-time changes. A per-kernel choice made after inspecting these results would be overfitting, so all results are reported and no outcome-guided selector is claimed.

Prior-work comparison. This evaluation does not reproduce Heo et al.'s instruction-level method, AVR target, or exact MiBench configuration. It therefore supports no direct performance or superiority comparison. The cited mixed effects motivate the general question only; our three neutral memory objects are not attributed to their implementation.

10 Future Work

Future work can broaden memory-object forms to structures and heap allocations, introduce alias-aware and interprocedural reasoning, and enrich range inference with ConstantRange or ScalarEvolution. The controlled investigation of allocation-equivalent safe widths showed that downstream LLVM optimization can favor different representations even when target allocation is unchanged. Future selection research should therefore model vectorization profitability, extraction and extension costs, control-flow structure, register pressure, and target-specific lowering effects, followed by independent validation across broader benchmark suites, LLVM releases, and target architectures. Additional evaluation can include randomized and property-based differential workloads, measured allocation layouts, embedded hardware energy experiments, and independent translation validation. These are extensions to the revised baseline, not features claimed by the current artifact.

11 Conclusion

The evaluation shows that static width/value evidence and target allocation benefit are distinct. In the controlled experiment, the DataLayout guard reduces 26 accepted transformations to 23, eliminates all three neutral cases, raises allocation-benefit precision from 88.46% to 100%, and retains all 67 B of common-set saving with 32/32 differential agreement. External validation on 17 MiBench programs repeats the selection result: precision rises from 95.24% to 100.00%, both neutral rewrites are removed, all 1252 B of common-set saving are retained, and all 16 executed comparisons agree. MiBench additionally shows an O2 IR delta change from +119 to +53, but executable sizes still increase slightly under both methods. The contribution is therefore improved transformation-selection precision and target-aware allocation reasoning, not demonstrated runtime, energy, physical-memory, or code-size improvement for the A/B executable-file comparison. The new allocation-equivalent-width experiment changes 17 object representations in 14 kernels with no allocation change: conventional widths improve static instruction counts in one kernel, tie in eleven, and regress in two, with aggregate increases of two instructions and 18 text bytes relative to minimum widths. Thus the minimum semantic width and a conventional width are not interchangeable optimization choices even when their storage cost is identical; neither the allocation guard nor simple rounding predicts their downstream profitability.

References

[1] Seonyeong Heo, Jiho Kim, Woohyeop Im, Jiyun Moon, and Daehee Jang. Bit-level compiler optimization for ultra low-power embedded systems. *Journal of Systems Architecture*, 168:103546, 2025. doi: 10.1016/j.sysarc.2025.103546.

[2] Scott A. Mahlke, Rajiv A. Ravindran, Michael S. Schlansker, Robert Schreiber, and Timothy Sherwood. Bitwidth cognizant architecture synthesis of custom hardware accelerators. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 20(11):1355–1371, 2001. doi: 10.1109/43.959864.

[3] V. Krishna Nandivada and Rajkishore Barik. Improved bitwidth-aware variable packing. *ACM Transactions on Architecture and Code Optimization*, 10(3):16:1–16:22, 2013. doi: 10.1145/2509420.2509427.

[4] Somashekaracharya G. Bhaskaracharya, Uday Bondhugula, and Albert Cohen. Automatic storage optimization for arrays. *ACM Transactions on Programming Languages and Systems*, 38(3):11:1–11:23, 2016. doi: 10.1145/2845078.

[5] Dong-U Lee, Altaf Abdul Gaffar, Ray C. C. Cheung, Oskar Mencer, Wayne Luk, and George A. Constantinides. Accuracy-guaranteed bit-width optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 25(10):1990–2000, 2006. doi: 10.1109/TCAD.2006.873887.

[6] Shalini Jain, Utpal Bora, Prateek Kumar, Vaibhav B. Sinha, Suresh Purini, and Ramakrishna Upadrasta. An analysis of executable size reduction by LLVM passes. *CSI Transactions on ICT*, 7(2):105–110, 2019. doi: 10.1007/s40012-019-00248-5.

[7] Stefano Cherubin and Giovanni Agosta. Tools for reduced preci-

sion computation: A survey. *ACM Computing Surveys*, 53(2):33:1–33:35, 2020. doi: 10.1145/3381039.

[8] Michael R. Jantz and Prasad A. Kulkarni. Analyzing and addressing false interactions during compiler optimization phase ordering. *Software: Practice and Experience*, 44(6):643–679, 2014. doi: 10.1002/spe.2176.

[9] Zhide Zhou, Zhilei Ren, Guojun Gao, and He Jiang. An empirical study of optimization bugs in GCC and LLVM. *Journal of Systems and Software*, 174:110884, 2021. doi: 10.1016/j.jss.2020.110884.

[10] Karine Even-Mendoza, Cristian Cadar, and Alastair F. Donaldson. CsmithEdge: More effective compiler testing by handling undefined behaviour less conservatively. *Empirical Software Engineering*, 27 (6):129, 2022. doi: 10.1007/s10664-022-10146-1.

[11] LLVM Project. LLVM Language Reference Manual. `https://releases.llvm.org/18.1.8/docs/LangRef.html`, 2026. Accessed 2026-09-13.

[12] LLVM Project. KnownBits and ValueTracking Infrastructure. `https://llvm.org/doxygen/classllvm_1_1KnownBits.html`, 2026. Accessed 2026-09-13.

[13] LLVM Project. DemandedBits Class Reference. `https://llvm.org/doxygen/classllvm_1_1DemandedBits.html`, 2026. Accessed 2026-09-13.

[14] LLVM Project. ConstantRange Class Reference. `https://llvm.org/doxygen/classllvm_1_1ConstantRange.html`, 2026. Accessed 2026-09-13.

[15] LLVM Project. ScalarEvolution Class Reference. `https://llvm.org/doxygen/classllvm_1_1ScalarEvolution.html`, 2026. Accessed 2026-09-13.

[16] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 238–252, 1977. doi: 10.1145/512950.512973.

[17] Mark N. Wegman and F. Kenneth Zadeck. Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems*, 13(2):181–210, 1991. doi: 10.1145/103135.103136.

[18] Mark Stephenson, Jonathan Babb, and Saman P. Amarasinghe. Bitwidth analysis with application to silicon compilation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 108–120, 2000. doi: 10.1145/349299.349317.

[19] Mihai Budiu, Majd Sakr, Kevin Walker, and Seth Copen Goldstein. Bitvalue inference: Detecting and exploiting narrow bitwidth computations. In *Euro-Par 2000 Parallel Processing*, volume 1900 of *Lecture Notes in Computer Science*, pages 969–979. Springer, 2000. doi: 10.1007/3-540-44520-X_136.

[20] Dong-U Lee, Altaf Abdul Gaffar, Oskar Mencer, and Wayne Luk. Minibit: Bit-width optimization via affine arithmetic. In *Proceedings of the 42nd Design Automation Conference*, pages 837–840, 2005. doi: 10.1145/1065579.1065799.

[21] LLVM Project. Scalar replacement of aggregates and memory promotion. `https://llvm.org/docs/Passes.html`, 2026. Accessed 2026-09-13.

[22] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 65–79, 2021. doi: 10.1145/3453483.3454030.

[23] LLVM Project. MLIR rationale: Data layout of non-primitive types. `https://mlir.llvm.org/docs/Rationale/Rationale/`, 2026. Accessed 2026-10-24; distinguishes logical bit width from target storage size.

[24] LLVM Project. Writing an LLVM pass with the new pass manager. `https://llvm.org/docs/WritingAnLLVMNewPMPass.html`, 2026. Accessed 2026-09-13.

[25] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization*, pages 75–86, 2004. doi: 10.1109/CGO.2004.1281665.

[26] Matthew R. Guthaus, Jeffrey S. Ringenberg, Dan Ernst, Todd M. Austin, Trevor Mudge, and Richard B. Brown. MiBench: A free, commercially representative embedded benchmark suite. In *Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization*, pages 3–14, 2001.

[27] LLVM Project. LLVM Test Suite. `https://github.com/llvm/llvm-test-suite`, 2026. MiBench source pinned at commit 4b2c892865cbbb05ee996c88794a58e17d3adb81; accessed 2026-09-13.